



Chapter 12

Recursion

จุดประสงค์

- เข้าใจความหมายและการทำงานของโปรแกรม Recursion ได้
- สามารถพัฒนาโปรแกรม Recursion อย่างง่ายได้

ทดสอบก่อนเรียน

- $\text{test}(2) = ?$

algorithm test(n)

int res

if (n == -3)

res = 5

else

res = test(n-1) - n

end if

return (res)

end test

$$\begin{aligned}\text{test}(2) &= \text{test}(2-1) - 2 \Rightarrow 5 \\ \text{test}(1) &= \text{test}(0) - 1 \Rightarrow 7 \\ 0 &= \text{test}(-1) - 0 \Rightarrow 8 \\ -1 &= \text{test}(-2) + 1 \Rightarrow 8 \\ -2 &= \text{test}(-3) + 2 \Rightarrow 7 \\ -3 &= 5\end{aligned}$$

อัลกอริทึมที่มีการทำซ้ำ (Repetitive Algorithm)

- การเขียนอัลกอริทึมที่มีการเรียกซ้ำทำได้ 2 รูปแบบ
 - Iteration : โปรแกรมที่มีการทำงานซ้ำตามเงื่อนไขที่กำหนด เช่น การใช้ for-loop, while-loop
 - Ex.

```
int i = 1;

while (i<5) {

    printf("Number : %d", i);

    i++;

}
```
 - Recursion : โปรแกรมย่อยที่มีการเรียกใช้ตัวเอง

Case Study : Factorial

$$\text{Factorial}(n) = \begin{cases} 1 & ,n=0 \\ n*(n-1)*(n-2)*...*3*2*1 & ,n>0 \end{cases}$$

- $\text{Factorial}(4) = 4*3*2*1 = 24$

Factorial : Iterative Solution

Algorithm iterativeFactorial (n)

Calculates the factorial of a number using a loop.

Pre n is the number to be raised factorially

Post n! is returned

1 set i to 1

2 set factN to 1

3 loop (i <= n)

1 set factN to factN * i

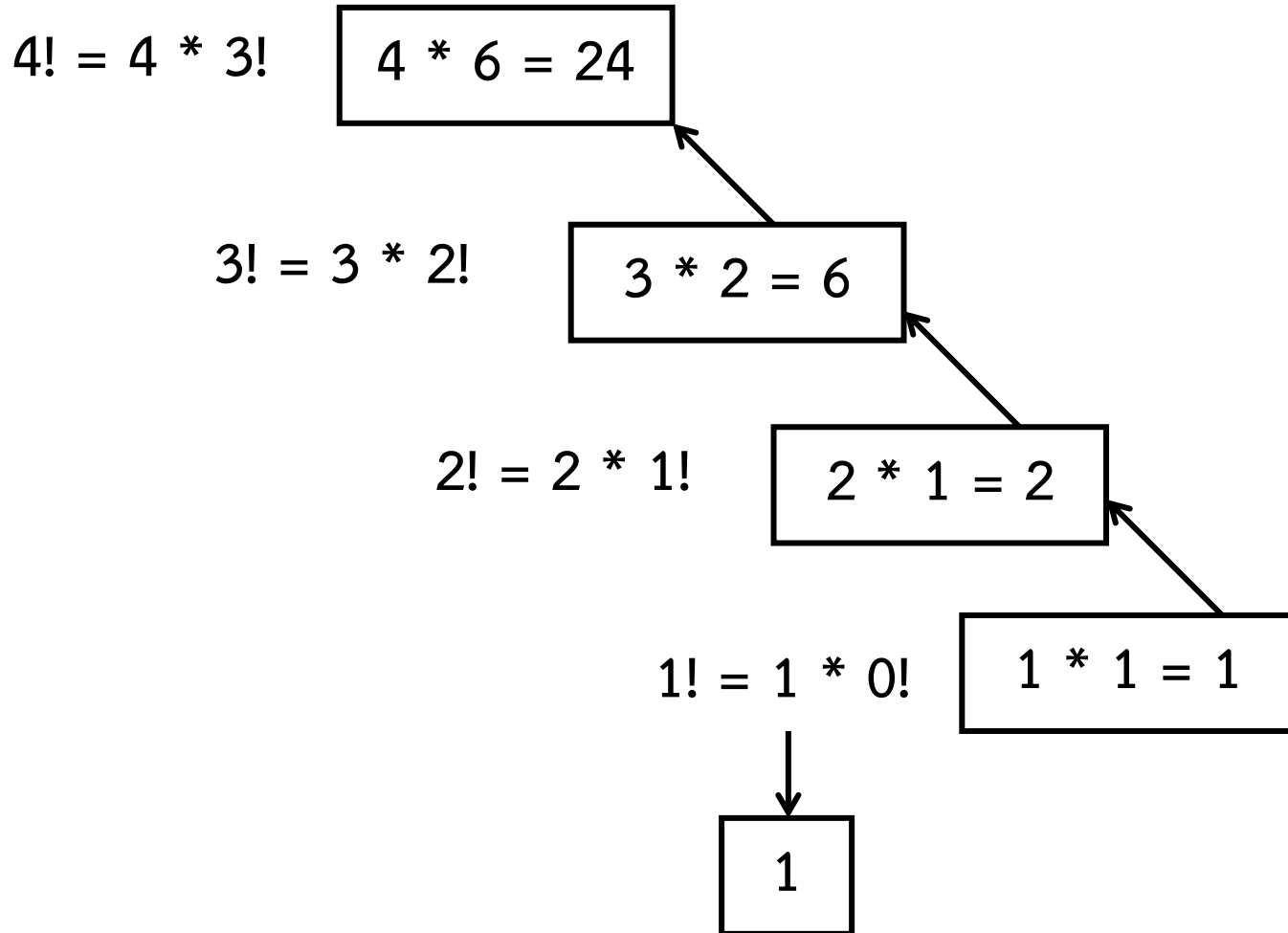
2 increment i

4 end loop

5 return factN

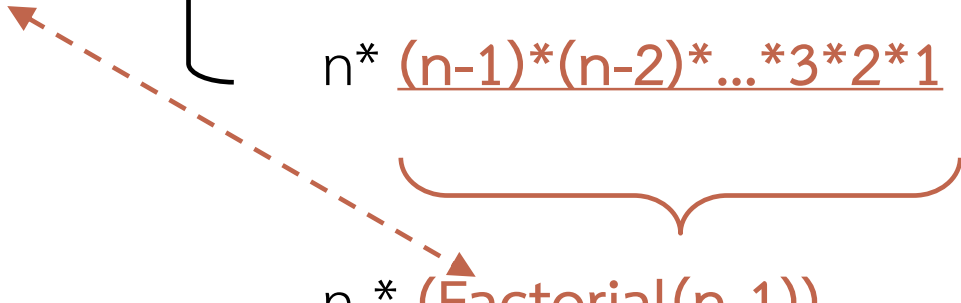
end iterativeFactorial

Case Study : Factorial(4)



Factorial : Recursion Defined

$$\text{Factorial}(n) = \begin{cases} 1 & ,n=0 \\ n * \underbrace{(n-1)*(n-2)*\dots*3*2*1}_{\text{Factorial}(n-1)} & ,n>0 \end{cases}$$



Factorial : Recursive Solution

Algorithm recursiveFactorial (n)

Calculates factorial of a number using recursion.

Pre n is the number being raised factorially

Post n! is returned

1 if (n equals 0)

1 return 1

2 else

1 return (n * recursiveFactorial (n - 1))

3 end if

end recursiveFactorial

```

program factorial
1 factN = recursiveFactorial(3)
2 print (factN)
end factorial

```

```

Algorithm recursiveFactorial (n)
1 if (n equals 0)
    1 return 1
2 else
    1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial

```

```

Algorithm recursiveFactorial (n)
1 if (n equals 0)
    1 return 1
2 else
    1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial

```

```

Algorithm recursiveFactorial (n)
1 if (n equals 0)
    1 return 1
2 else
    1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial

```

```

Algorithm recursiveFactorial (n)
1 if (n equals 0)
    1 return 1
2 else
    1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial

```

Factorial :

Calling a Recursive
Algorithm

The Design Methodology

- การเรียกตัวเองซ้ำ จะประกอบด้วยการทำงาน 2 ส่วน
 - ส่วนที่ทำการหยุดการเรียกซ้ำ -> *base case*
 - ส่วนที่ทำการเรียกซ้ำ -> *general case*
- Factorial
 - *Base case* : factorial(0)
 - *General case* : $n * \text{factorial}(n-1)$

Rules for designing

- การเรียกตัวเองซ้ำ จะเรียกตัวเองไปเรื่อยๆ เพื่อลดขนาดของปัญหาลงจนมีการหยุดการเรียกซ้ำ
- การเขียนโปรแกรมย่อยที่มีการเรียกตัวเองซ้ำ จะต้อง
 - กำหนดส่วนที่ทำให้หยุดการเรียกซ้ำ
 - กำหนดส่วนที่ทำการเรียกซ้ำ

Limitations of Recursion

- เราไม่ควรใช้ลักษณะการเรียกตัวเองซ้ำ เมื่อ
 - อัลกอริทึมหรือโครงสร้างข้อมูลที่ใช้ ไม่เหมาะกับการเรียกตัวเองซ้ำ
 - การเรียกตัวเองซ้ำ ทำให้ยุ่งยากมากขึ้น (โค้ดยาวขึ้น เข้าใจยาก)
 - ใช้เนื้อที่และเวลาในการทำงานมาก

Print Reverse Example

```
Algorithm printReverse (data)
Print keyboard data in reverse.
  Pre  nothing
  Post data printed in reverse
1 if (end of input)
  1  return
2 end if
3 read data
4 printReverse (data)
Have reached end of input: print nodes
5 print data
6 return
end printReverse
```


Print Reverse Example (cont.)

- พิจารณาว่าการทำงานนี้ควรใช้การเรียกตัวเองซ้ำหรือไม่
 - อัลกอริทึมนี้ไม่เหมาะกับการเรียกตัวเองซ้ำ เนื่องจากไม่ใช่ logarithmic algorithm
 - อัลกอริทึมนี้เข้าใจได้ยาก
 - ประสิทธิภาพการทำงานของอัลกอริทึมคิดเป็น $O(n)$ (ซึ่งถือว่าช้า)
- *การทำงานนี้ไม่ควรใช้การเรียกตัวเองซ้ำ*



Recursive Example

- Greatest Common Divisor
- Fibonacci number
- Towers of Hanoi

Greatest Common Divisor

$$\text{gcd}(a, b) = \begin{cases} a & \text{if } b = 0 \\ b & \text{if } a = 0 \\ \text{gcd}(b, a \bmod b) & \text{otherwise} \end{cases}$$

$$\begin{aligned} \text{gcd}(10, 25) &= \text{gcd}(25, 10) \\ &= \text{gcd}(10, 5) \\ &= \text{gcd}(5, 0) \\ &= 5 \end{aligned}$$

$$\begin{cases} \text{if } b = 0 \\ \text{if } a = 0 \\ \text{otherwise} \end{cases}$$

Greatest Common Divisor (cont.)

Algorithm gcd (a, b)

Calculates greatest common divisor using the Euclidean algorithm.

Pre a and b are positive integers greater than 0

Post greatest common divisor returned

```
1 if (b equals 0)
  1 return a
2 end if
3 if (a equals 0)
  2 return b
4 end if
5 return gcd (b, a mod b)
end gcd
```

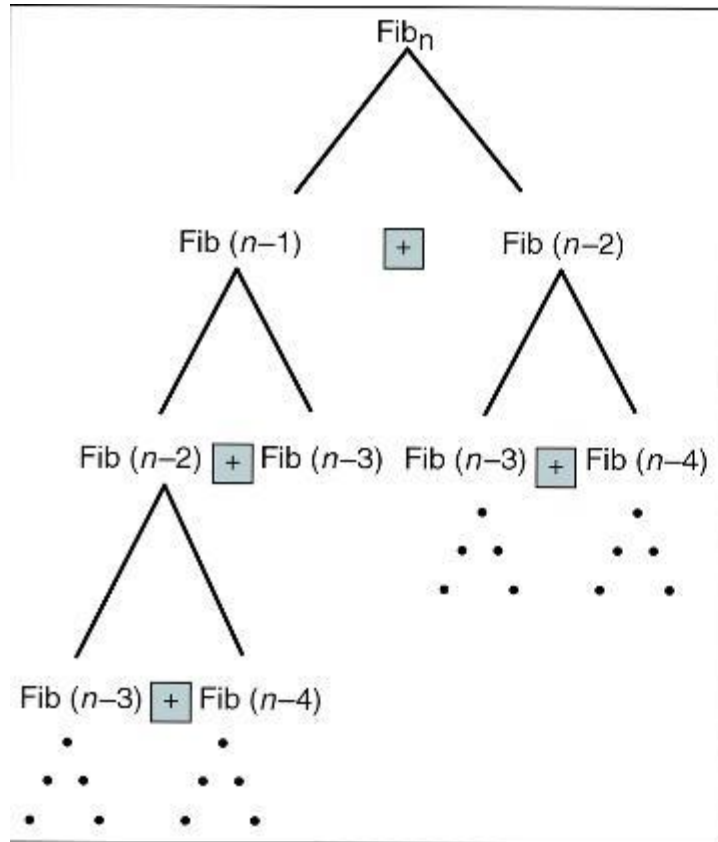
Fibonacci Numbers

0 1 1 2 3 5 8 13 21
 $f(0)$ $f(1)$ $f(2)$ $f(3)$ $f(4)$ $f(5)$

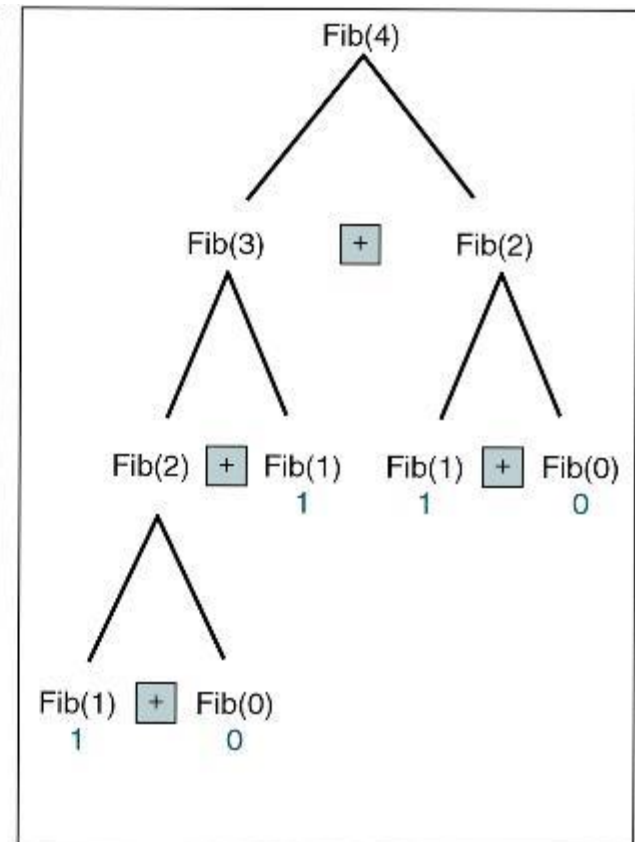
$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{otherwise} \end{cases}$$

$$\begin{aligned} f(4) &= f(3) + f(2) \\ &= (f(2) + f(1)) + (f(1) + f(0)) \\ &= (1+1) + 1 = 3 \end{aligned}$$

Fibonacci Numbers (cont.)



(a) $\text{Fib}(n)$



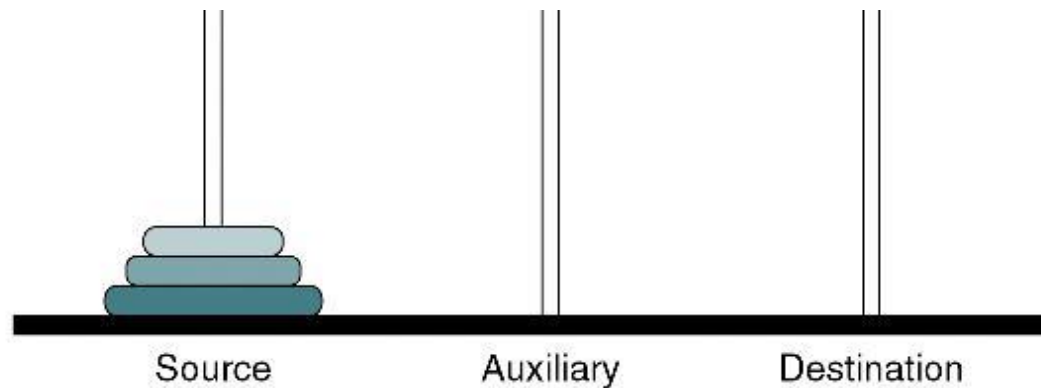
(b) $\text{Fib}(4)$

Fibonacci Numbers (cont.)

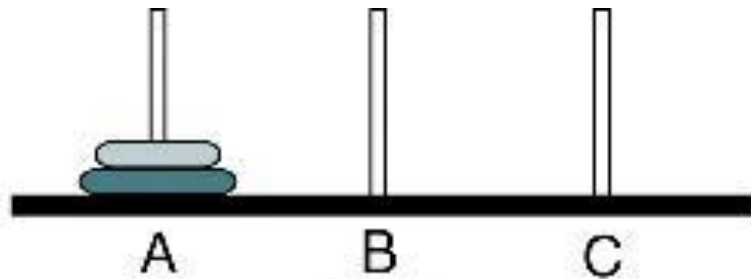
fib(<i>n</i>)	Calls	fib(<i>n</i>)	Calls
1	1	11	287
2	3	12	465
3	5	13	753
4	9	14	1219
5	15	15	1973
6	25	20	21,891
7	41	25	242,785
8	67	30	2,692,573
9	109	35	29,860,703
10	177	40	331,160,281

The Towers of Hanoi

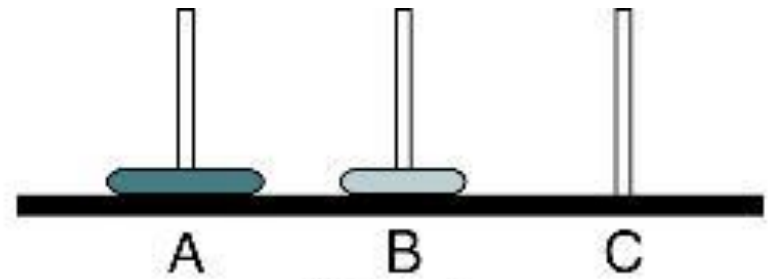
- ให้ทำการย้ายแผ่นดิสก์จากแท่น Source ไปยังแท่น Destination โดยที่
 - ดิสก์แผ่นเล็กต้องอยู่บนดิสก์แผ่นใหญ่เสมอ และสามารถย้ายแผ่นดิสก์ได้ครั้งละหนึ่งแผ่นเท่านั้น
 - ใช้แท่น Auxiliary ช่วยพักแผ่นดิสก์ชั่วคราว (ช่วยย้ายแผ่นดิสก์ไปยังแท่น Destination)



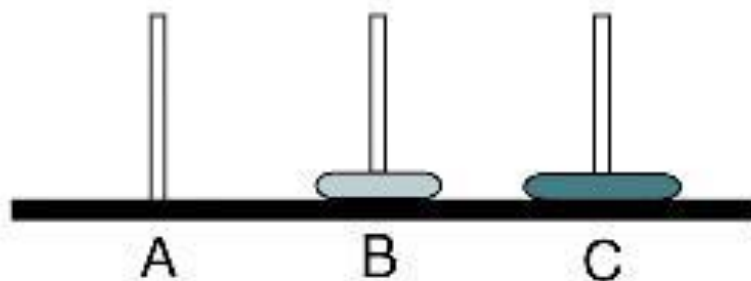
Towers Solution for Two Disks



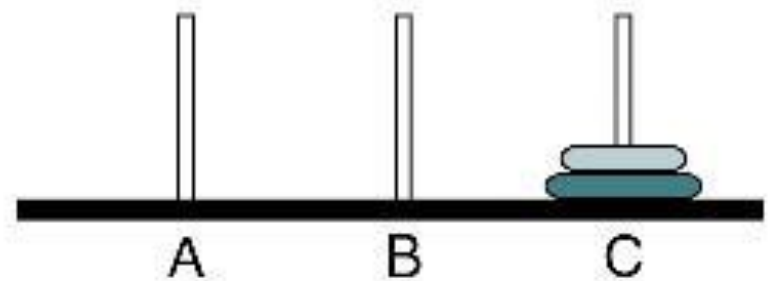
Start



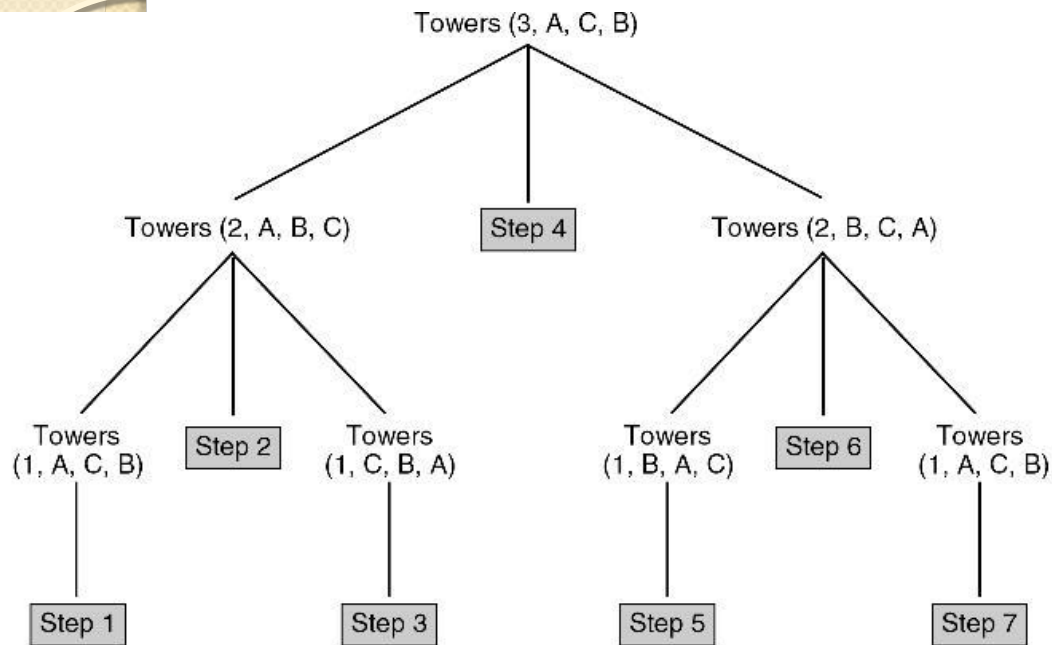
Step 1



Step 2

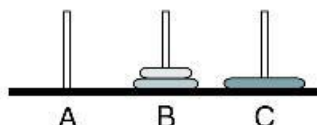
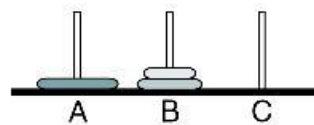
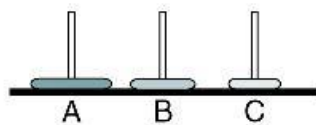
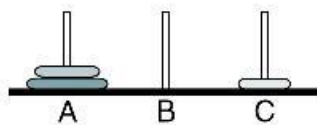
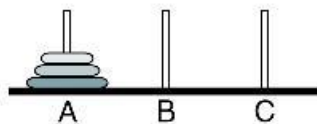


Step 3

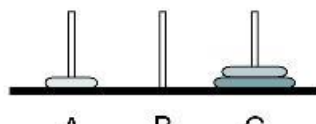
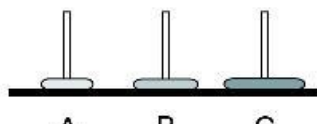


Towers Solution for Three Disks

Source
 Tower (3, A, B, C)
 Dish
 Des
 Alex



Move one disk from source to destination.



Towers Solution for Three Disks

1. Move $n-1$ disks from source to auxiliary.
2. Move one disk from source to destination.
3. Move $n-1$ disks from auxiliary to destination.

The Towers of Hanoi

```
Algorithm towers (numDisks, source, dest, auxiliary)
  Recursively move disks from source to destination.
    Pre  numDisks is number of disks to be moved
         source, destination, and auxiliary towers given
    Post steps for moves printed
1  print("Towers: ", numDisks, source, dest, auxiliary)
2  if (numDisks is 1)
    1  print ("Move from ", source, " to ", dest)
3  else
    1  towers (numDisks - 1, source, auxiliary, dest, step)
    2  print ("Move from " source " to " dest)
    3  towers (numDisks - 1, auxiliary, dest, source, step)
4  end if
end towers
```

The Towers of Hanoi

Calls:

Towers (3, A, C, B)

Towers (2, A, B, C)

Towers (1, A, C, B)

Towers (1, C, B, A)

Towers (2, B, C, A)

Towers (1, B, A, C)

Towers (1, A, C, B)

Output:

Move from A to C

Move from A to B

Move from C to B

Move from A to C

Move from B to A

Move from B to C

Move from A to C

Quiz 1

Algorithm quiz1(n)

 int r1

 if (n == 1)

 r1 = 12

 else

 r1 = n + quiz1(n/2)

 end if

 return (r1)

end quiz1

quiz1(32) = ?

Quiz 2

Algorithm quiz2(n)

if (n==8)

println("Go Back!")

else

println(n + " Hello!")

quiz2(n+2)

println(n + " Bye!")

end if

end quiz2

quiz2(0) = ?

Quiz 3

Algorithm quiz3(x)

if (x<5)

return (3*x)

else

return (2*quiz3(x-5)+7)

end if

println("Finish!!")

return 1

end quiz3

quiz3(4) = ?

quiz3(10) = ?

quiz3(17) = ?